

SmartSpend

Personal Finance & Budget Analytics Dashboard — Internship Project Report

Name: Bhavana Gajendran
Duration: 2-Week Project Phase

Program: Elevate Labs Internship (Web Development)
Repository:
github.com/bhavanagajendran03-ui/SmartSpend

Abstract

SmartSpend is a browser-based personal finance dashboard that lets a user log income and expenses, visualize spending patterns through charts, set savings goals and category-wise budget limits, and receive automated, rule-based insights into their financial habits. The project was built as an expanded version of the provided “Budget Planner with Data Visualizations” brief, adding authentication, analytics, and usability features that mirror a production-grade fintech application while remaining achievable within a two-week internship timeline.

Introduction

Manually tracking income and expenses across notes, spreadsheets, or memory makes it difficult to notice spending patterns or stay within a budget. SmartSpend addresses this by giving users a single dashboard to record every transaction, see where their money goes through visual charts, and get warned before they overspend in a category. The application also introduces savings goals and a financial health score, turning raw transaction data into actionable, easy-to-read feedback — the kind of feature set expected from a modern budgeting product.

Tools Used

Layer	Technology
Frontend structure & styling	HTML5, CSS3 (custom design system)
Application logic	JavaScript (ES6+), modular vanilla-JS architecture
Data visualization	Chart.js (pie, bar, and line charts)
Authentication	Local email/password authentication layer, structured to mirror Firebase Authentication
Data persistence	Browser localStorage, wrapped in a Firestore-style collection API
Version control & hosting	Git, GitHub, GitHub Pages (live deployment)
Development environment	Visual Studio Code, VS Code Live Server

Steps Involved in Building the Project

- 1. Data modeling** — Designed the transaction, savings-goal, and budget-limit data structures, and built a small persistence layer (store.js) exposing Firestore-style methods (getCollection, addDoc, updateDoc, deleteDoc) over localStorage.
- 2. Authentication** — Implemented sign-up, login, logout, and a password-reset flow, with each user's data isolated by user ID, plus a seeded demo account for quick evaluation.
- 3. Core CRUD screens** — Built forms and list views to add, edit, and delete income and expense transactions, each with category, date, payment method, and description.
- 4. Data visualization** — Integrated Chart.js to render a category-wise spending doughnut chart, a 6-month income-vs-expense bar chart, and a 30-day spending trend line chart.

- 5. Filtering & search** — Added filters by transaction type, category, and month, plus free-text search across category, description, and payment method.
- 6. Goals & budgets** — Implemented savings goals with progress bars and deadlines, and per-category budget limits with 80%-warning and over-limit alerts.
- 7. Insights & health score** — Wrote rule-based logic to generate spending insights (month-over-month category changes, savings-rate flags, potential savings estimates) and a transparent 0–100 financial health score combining savings rate, budget adherence, goal progress, and expense-to-income ratio.
- 8. Reports** — Built a monthly summary view with income, expenses, savings rate, and highest/lowest expense categories, alongside a month-to-month comparison with directional indicators.
- 9. Polish & accessibility** — Added a dark-mode toggle, a fully responsive layout for mobile and tablet, and CSV export of the currently filtered transaction list.
- 10. Version control & deployment** — Initialized a Git repository, pushed the project to GitHub, and deployed it live via GitHub Pages for demonstration.

Conclusion

SmartSpend successfully delivers a complete personal finance workflow — authentication, CRUD, data visualization, filtering, goal tracking, budget alerts, and automated insights — within a single, responsive web application. Given more time, the project could be extended by replacing the local persistence layer with real Firebase Authentication and Firestore for true multi-device sync, adding recurring/scheduled transactions, multi-currency support, and shared/family budgets. The codebase was deliberately structured so that this migration only touches two files (`store.js` and `auth.js`), making it a natural next step beyond this internship phase.